ОТЧЁТ

по лабораторной работе № 2

Эмпирический анализ временной сложности алгоритмов сортировки. Часть 1: квадратичные алгоритмы (сортировка обменом и сортировка вставками)

Дисциплина: Теория алгоритмов

Направление подготовки: 38.03.05 «Бизнес-информатика»

Вариант 19

Выполнила: студентка 2 курса, группа БИ2-1
Ситникова Ольга

2026

1. Цель работы и задание варианта

Цель работы: освоить методику экспериментального исследования временной сложности алгоритмов и сопоставить полученные эмпирические зависимости с теоретическими асимптотическими оценками на примере простых алгоритмов сортировки.

Задание варианта 19. Сравнить время работы сортировки пузырьком (с флагом) и сортировки вставками на почти упорядоченных целочисленных массивах (тип N: упорядоченный массив, в котором выполнено n/20 случайных перестановок пар) со значениями из диапазона [0; 50 000]. Размеры массивов n = 1000, 2000, 3000, 4000, 5000, 6000. Каждое измерение повторить k = 3 раза и взять медиану.

Дополнительное задание M5. Реализовать шейкерную сортировку (двунаправленный пузырёк) и сравнить её с классической сортировкой пузырьком.

№	Размеры n	Тип данных	Диапазон	k	Доп. задание
19	1000, 2000, 3000, 4000, 5000, 6000	N (почти упорядоченные)	[0; 50 000]	3	M5

2. Описание алгоритмов и теоретические оценки сложности

Сортировка обменом («пузырьком»). Массив просматривается слева направо, соседние элементы сравниваются и при нарушении порядка меняются местами. После i-го прохода i наибольших элементов занимают окончательные позиции в конце массива. Если за проход не выполнено ни одного обмена (флаг *swapped*), массив упорядочен и работа завершается досрочно: на упорядоченных данных достаточно одного прохода.

Шейкерная сортировка (M5). Модификация пузырька: проходы выполняются попеременно слева направо (максимум уходит в конец) и справа налево (минимум уходит в начало). Границы неупорядоченной части сужаются с обеих сторон. Алгоритм также завершается, если за проход не было обменов.

Сортировка вставками. Массив делится на отсортированную (левую) и неотсортированную (правую) части. Очередной элемент (ключ) вставляется в нужное место отсортированной части: бóльшие элементы сдвигаются вправо. Число сдвигов равно числу инверсий массива, поэтому на почти упорядоченных данных алгоритм быстрее, чем на случайных.

Все три алгоритма работают «на месте» (O(1) дополнительной памяти) и устойчивы.

Таблица 1. Теоретические оценки

Алгоритм	Лучший случай	Средний случай	Худший случай	Доп. память	Устойчивость
Пузырьком (с флагом)	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$	$O(1)$	Да
Шейкерная (с флагом)	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$	$O(1)$	Да
Вставками	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$	$O(1)$	Да

Для квадратичного алгоритма $T(n) = \Theta(n^2)$, поэтому при увеличении n в 2 раза время должно возрастать примерно в 4 раза, а отношение $T(n)/n^2$ должно быть приблизительно постоянным. Именно это проверяется в работе.

3. Блок-схемы алгоритмов

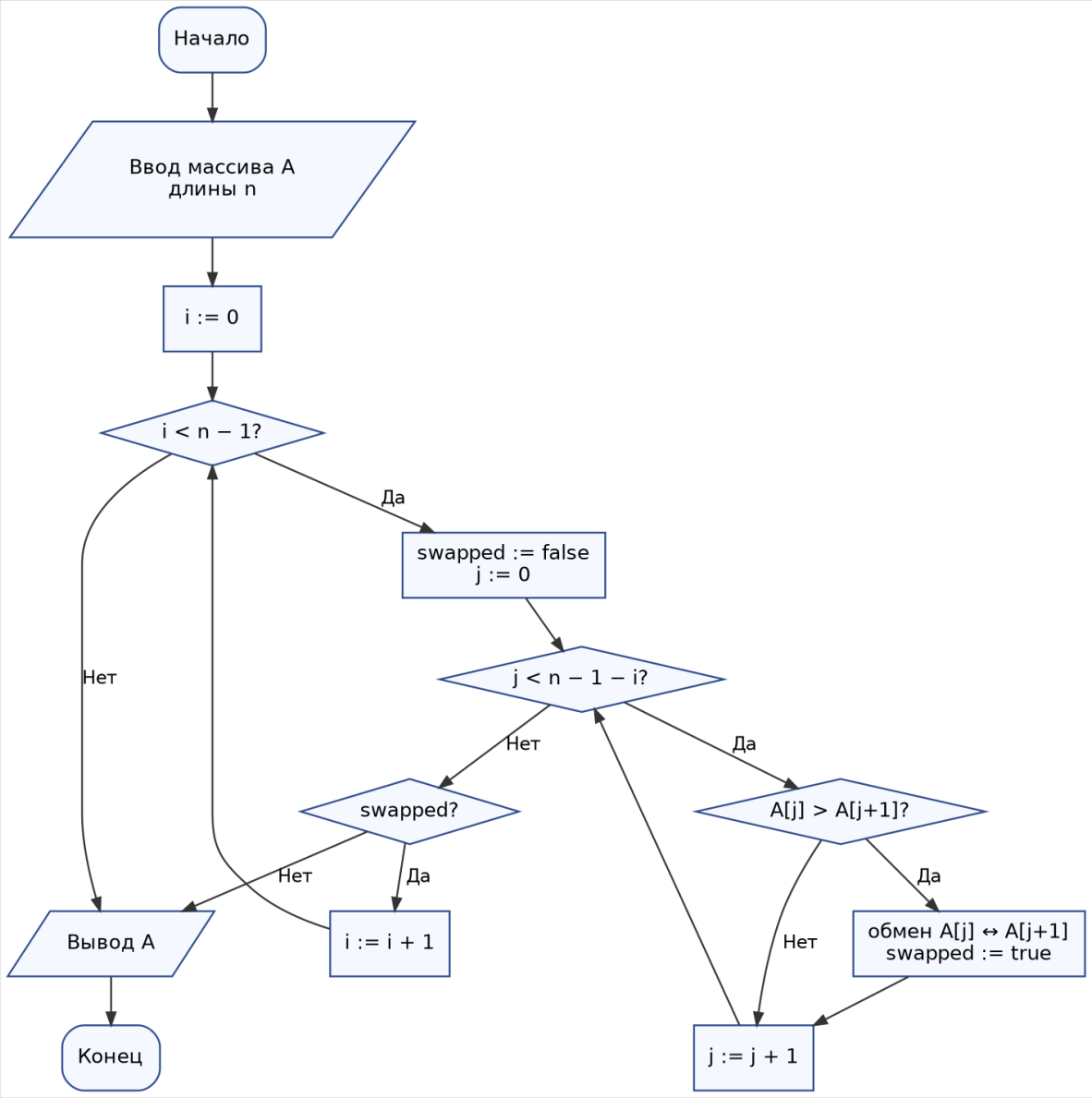


Рис. 1. Блок-схема сортировки пузырьком (с флагом)

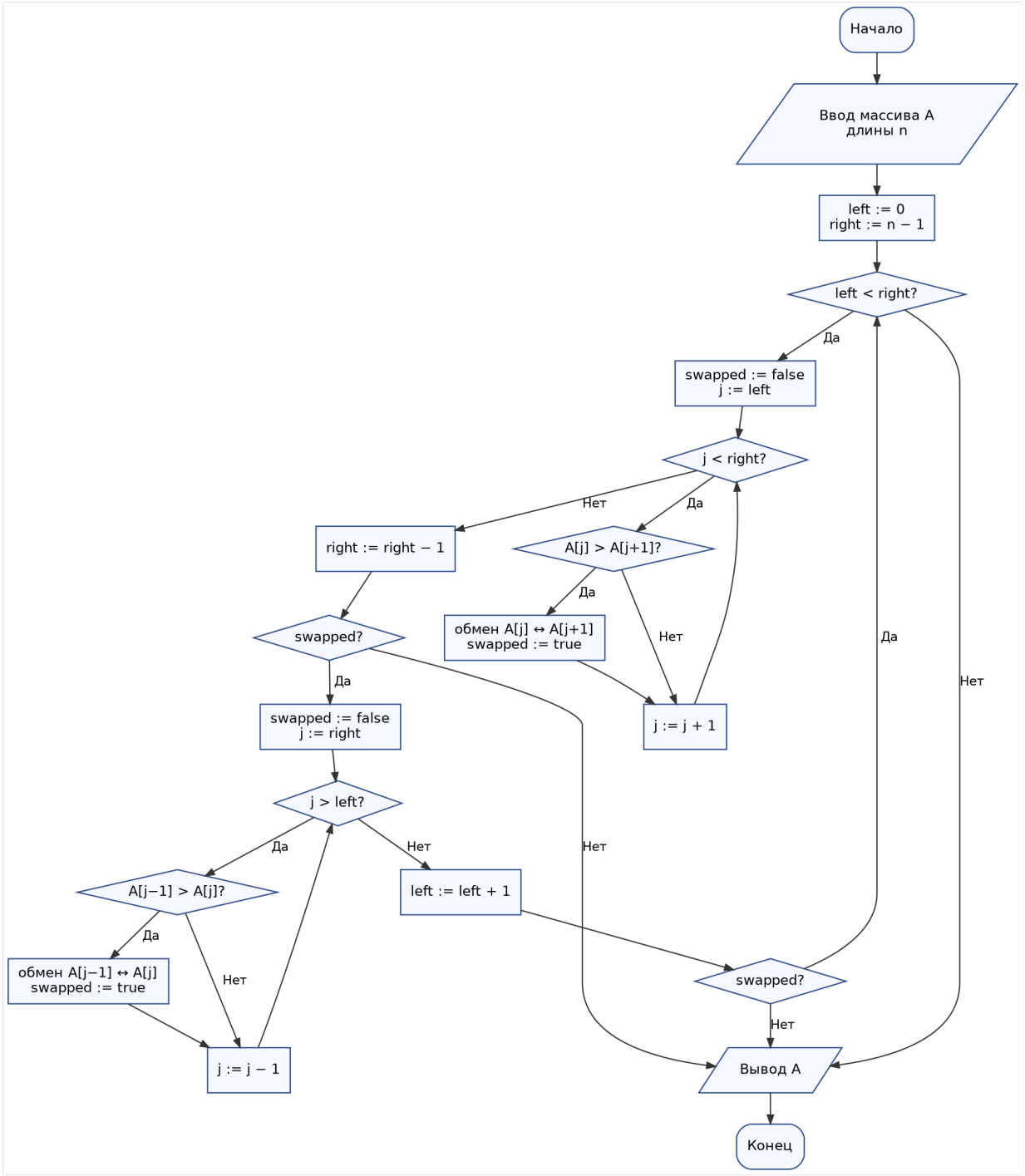


Рис. 2. Блок-схема шейкерной сортировки (M5)

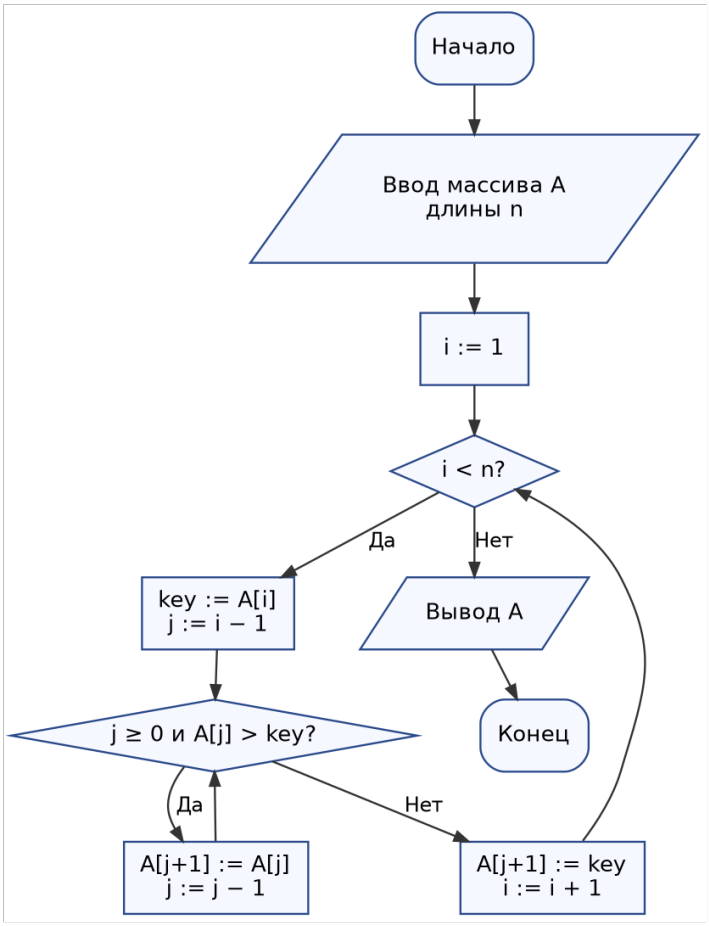


Рис. 3. Блок-схема сортировки вставками

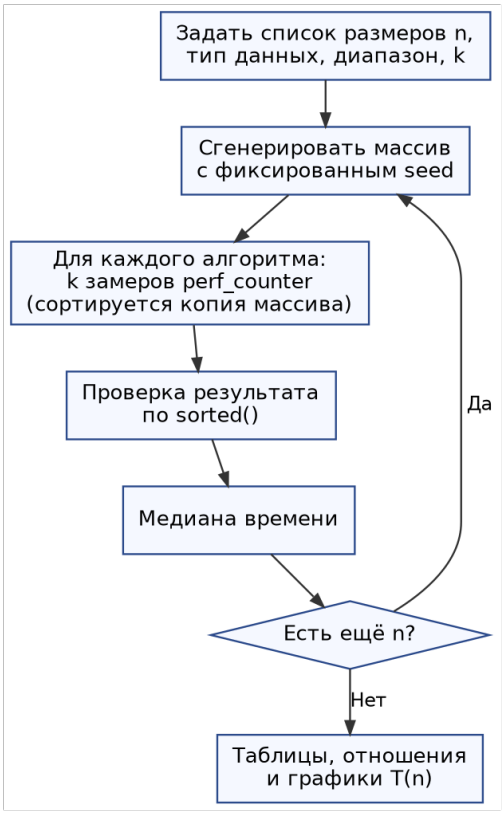


Рис. 4. Блок-схема экспериментального стенда

4. Листинг программы

Программа находится в файле `lr2.py`; запуск: `python  lr2.py` (для построения графиков нужен пакет `matplotlib`).

```
"""
Лабораторная работа № 2. Вариант 19.
Эмпирический анализ временной сложности квадратичных сортировок.

Автор: Ситникова Ольга, группа БИ2-1.

Исходные данные варианта 19:
    размеры массивов n = 1000, 2000, 3000, 4000, 5000, 6000
    тип данных: N (почти упорядоченные: n/20 случайных перестановок пар)
    диапазон значений: [0; 50 000]
    число повторов k = 3 (берётся медиана)
    дополнительное задание М5: шейкерная сортировка (двунаправленный пузырёк)
    и её сравнение с классической сортировкой пузырьком

Запуск: python lr2.py   (для графиков нужен пакет matplotlib)
"""
import platform
import random
import statistics
import sys
import time

# ---------------------------------------------------------------------------
# Параметры варианта 19
# ---------------------------------------------------------------------------
SIZES = [1000, 2000, 3000, 4000, 5000, 6000]
KIND = "nearly_sorted"
LO, HI = 0, 50_000
REPEATS = 3
SEED = 42

if hasattr(sys.stdout, "reconfigure"):      # корректный вывод кириллицы в Windows
    sys.stdout.reconfigure(encoding="utf-8")


# ---------------------------------------------------------------------------
# Генерация данных и замер времени
# ---------------------------------------------------------------------------
def generate_data(n, kind="random", lo=0, hi=100_000, seed=SEED):
    """Генерирует массив длины n заданного типа (воспроизводимо, по seed)."""
    rng = random.Random(seed + n)
    data = [rng.randint(lo, hi) for _ in range(n)]
    if kind == "sorted":
        data.sort()
    elif kind == "reversed":
        data.sort(reverse=True)
    elif kind == "nearly_sorted":
        data.sort()
        for _ in range(max(1, n // 20)):              # ~5 % случайных перестановок
            i, j = rng.randrange(n), rng.randrange(n)
            data[i], data[j] = data[j], data[i]
    return data


def measure(sort_func, data, repeats=REPEATS):
    """Медианное время работы sort_func на данных data, с.

    Каждый раз сортируется копия data (функции исходный массив не меняют).
    Корректность результата проверяется по sorted() вне замера времени.
    """
    times = []
    for _ in range(repeats):
        start = time.perf_counter()
        result = sort_func(data)
        times.append(time.perf_counter() - start)
        assert result == sorted(data), f"{sort_func.__name__}: ошибка сортировки"
    return statistics.median(times)


# ---------------------------------------------------------------------------
# Алгоритмы сортировки (исходный массив не изменяют)
# ---------------------------------------------------------------------------
def bubble_sort(arr):
    """Сортировка пузырьком с флагом досрочного выхода."""
    a = arr.copy()
    n = len(a)
    for i in range(n - 1):
        swapped = False
        for j in range(n - 1 - i):
            if a[j] > a[j + 1]:
                a[j], a[j + 1] = a[j + 1], a[j]
```

```
                swapped = True
        if not swapped:
            break
    return a


def shaker_sort(arr):
    """Шейкерная сортировка (двунаправленный пузырёк) с флагом выхода.

    Проход вправо выносит максимум в конец, проход влево выносит минимум
    в начало; границы неупорядоченной части [left, right] сужаются с обеих сторон.
    """
    a = arr.copy()
    left, right = 0, len(a) - 1
    while left < right:
        swapped = False
        for j in range(left, right):              # проход слева направо
            if a[j] > a[j + 1]:
                a[j], a[j + 1] = a[j + 1], a[j]
                swapped = True
        right -= 1
        if not swapped:
            break
        swapped = False
        for j in range(right, left, -1):          # проход справа налево
            if a[j - 1] > a[j]:
                a[j - 1], a[j] = a[j], a[j - 1]
                swapped = True
        left += 1
        if not swapped:
            break
    return a


def insertion_sort(arr):
    """Сортировка вставками."""
    a = arr.copy()
    for i in range(1, len(a)):
        key = a[i]
        j = i - 1
        while j >= 0 and a[j] > key:
            a[j + 1] = a[j]
            j -= 1
        a[j + 1] = key
    return a


# ---------------------------------------------------------------------------
# Версии с подсчётом операций (только для анализа, в замере времени не участвуют)
# ---------------------------------------------------------------------------
def bubble_sort_counted(arr):
    """Возвращает (сравнений, обменов, проходов) для сортировки пузырьком."""
    a = arr.copy()
    n = len(a)
    comparisons = swaps = passes = 0
    for i in range(n - 1):
        swapped = False
        passes += 1
        for j in range(n - 1 - i):
            comparisons += 1
            if a[j] > a[j + 1]:
                a[j], a[j + 1] = a[j + 1], a[j]
                swaps += 1
                swapped = True
        if not swapped:
            break
    return comparisons, swaps, passes


def shaker_sort_counted(arr):
    """Возвращает (сравнений, обменов, проходов) для шейкерной сортировки."""
    a = arr.copy()
    left, right = 0, len(a) - 1
    comparisons = swaps = passes = 0
    while left < right:
        swapped = False
        passes += 1
        for j in range(left, right):
            comparisons += 1
            if a[j] > a[j + 1]:
                a[j], a[j + 1] = a[j + 1], a[j]
                swaps += 1
                swapped = True
        right -= 1
        if not swapped:
            break
        swapped = False
```

```
        passes += 1
        for j in range(right, left, -1):
            comparisons += 1
            if a[j - 1] > a[j]:
                a[j - 1], a[j] = a[j], a[j - 1]
                swaps += 1
                swapped = True
        left += 1
        if not swapped:
            break
    return comparisons, swaps, passes


def insertion_sort_counted(arr):
    """Возвращает (сравнений, сдвигов, проходов) для сортировки вставками."""
    a = arr.copy()
    comparisons = shifts = 0
    for i in range(1, len(a)):
        key = a[i]
        j = i - 1
        while j >= 0:
            comparisons += 1
            if a[j] <= key:
                break
            a[j + 1] = a[j]
            shifts += 1
            j -= 1
        a[j + 1] = key
    return comparisons, shifts, max(0, len(a) - 1)


# ---------------------------------------------------------------------------
# Вывод и графики
# ---------------------------------------------------------------------------
def print_system_info():
    """Характеристики вычислительной системы (для отчёта)."""
    print("Характеристики системы:")
    print(f"  ОС:        {platform.platform()}")
    print(f"  Процессор: {platform.processor() or 'не определён'}")
    print(f"  Python:    {platform.python_version()}")
    print()


def print_tables(results):
    names = list(results)
    print(f"Тип данных: почти упорядоченные, диапазон [{LO}; {HI}], "
          f"k = {REPEATS}, seed = {SEED}\n")

    print("Таблица 1. Медианное время сортировки, с")
    print(f"{'n':>6}" + "".join(f"{name:>14}" for name in names))
    for idx, n in enumerate(SIZES):
        print(f"{n:>6}" + "".join(f"{results[name][idx]:>14.4f}" for name in names))
    print()

    print("Таблица 2. Отношение T(n)/n^2 (x 1e-9, с)")
    print(f"{'n':>6}" + "".join(f"{name:>14}" for name in names))
    for idx, n in enumerate(SIZES):
        print(f"{n:>6}" + "".join(f"{results[name][idx] / n**2 * 1e9:>14.2f}" for name in names))
    print()

    print("Таблица 3. Проверка квадратичного роста: T(2n)/T(n)")
    print(f"{'Переход':>14}" + "".join(f"{name:>14}" for name in names) + f"{'Теория':>10}")
    for n in SIZES:
        if 2 * n in SIZES:
            i, j = SIZES.index(n), SIZES.index(2 * n)
            row = f"{f'{n} -> {2 * n}':>14}"
            row += "".join(f"{results[name][j] / results[name][i]:>14.2f}" for name in names)
            print(row + f"{4:>10}")
    print()

    print("Таблица 4. Отношение времени (пузырёк / вставки) и (пузырёк / шейкер)")
    print(f"{'n':>6}{'пузырёк/вставки':>18}{'пузырёк/шейкер':>18}")
    for idx, n in enumerate(SIZES):
        b, s, ins = (results["Пузырьком"][idx], results["Шейкерная"][idx],
                     results["Вставками"][idx])
        print(f"{n:>6}{b / ins:>18.2f}{b / s:>18.2f}")
    print()


def count_operations():
    """Число сравнений, обменов (сдвигов) и проходов для каждого размера n."""
    counters = {
        "Пузырьком": bubble_sort_counted,
        "Шейкерная": shaker_sort_counted,
        "Вставками": insertion_sort_counted,
    }
    return {name: [f(generate_data(n, KIND, LO, HI)) for n in SIZES]
```

```
            for name, f in counters.items()}


def print_operations(ops):
    names = list(ops)
    for title, idx in (("Таблица 5. Число сравнений", 0),
                       ("Таблица 6. Число обменов (для вставок — сдвигов)", 1),
                       ("Таблица 7. Число проходов по массиву", 2)):
        print(title)
        ref_head = f"{'n^2/2':>14}" if idx == 0 else ""
        print(f"{'n':>6}" + "".join(f"{name:>14}" for name in names) + ref_head)
        for k, n in enumerate(SIZES):
            ref = f"{n * n // 2:>14}" if idx == 0 else ""
            print(f"{n:>6}" + "".join(f"{ops[name][k][idx]:>14}" for name in names) + ref)
        print()


def make_plots(results):
    try:
        import matplotlib
        matplotlib.use("Agg")
        import matplotlib.pyplot as plt
    except ImportError:
        print("matplotlib не установлен: графики не построены (pip install matplotlib)")
        return

    # График 1: пузырёк и вставки на одних осях
    plt.figure(figsize=(8, 5))
    for name in ("Пузырьком", "Вставками"):
        plt.plot(SIZES, results[name], marker="o", label=name)
    plt.xlabel("Размер массива n")
    plt.ylabel("Время, с")
    plt.title("Почти упорядоченные данные: пузырёк и вставки")
    plt.grid(True)
    plt.legend()
    plt.tight_layout()
    plt.savefig("lr2_plot.png", dpi=150)
    plt.close()

    # График 2 (М5): пузырёк и шейкерная сортировка
    plt.figure(figsize=(8, 5))
    for name in ("Пузырьком", "Шейкерная"):
        plt.plot(SIZES, results[name], marker="o", label=name)
    plt.xlabel("Размер массива n")
    plt.ylabel("Время, с")
    plt.title("М5. Сортировка пузырьком и шейкерная сортировка")
    plt.grid(True)
    plt.legend()
    plt.tight_layout()
    plt.savefig("lr2_plot_m5.png", dpi=150)
    plt.close()
    print("Графики сохранены: lr2_plot.png, lr2_plot_m5.png")


def main():
    algorithms = {
        "Пузырьком": bubble_sort,
        "Шейкерная": shaker_sort,
        "Вставками": insertion_sort,
    }
    print_system_info()
    results = {name: [] for name in algorithms}
    for n in SIZES:
        data = generate_data(n, KIND, LO, HI)        # один массив для всех алгоритмов
        for name, func in algorithms.items():
            results[name].append(measure(func, data, REPEATS))
    print_tables(results)
    print_operations(count_operations())
    make_plots(results)


if __name__ == "__main__":
    main()
```

5. Характеристики вычислительной системы

Параметр	Значение
Операционная система	Linux-6.18.44-fc-v180-x86_64-with-glibc2.39
Процессор	Intel Xeon, 2,10 ГГц (виртуальная машина, 1 vCPU)
Оперативная память	4 ГБ
Интерпретатор	Python 3.12.3, matplotlib 3.10
Таймер	time.perf_counter()

Абсолютные значения времени зависят от системы; на другом компьютере они будут отличаться, но соотношения между алгоритмами и характер роста T(n) должны сохраниться.

6. Результаты измерений

Условия: массивы типа N, диапазон [0; 50 000], k = 3, медиана; для всех алгоритмов использовался один и тот же массив (seed = 42), результат каждой сортировки проверялся по `sorted()`.

Таблица 2. Медианное время сортировки почти упорядоченных массивов, с

n	Пузырьком	Шейкерная	Вставками
1000	0,0177	0,0031	0,0016
2000	0,0799	0,0154	0,0072
3000	0,1663	0,0282	0,0127
4000	0,2796	0,0472	0,0225
5000	0,4539	0,0768	0,0367
6000	0,6624	0,1027	0,0491

Таблица 3. Отношение времени работы алгоритмов

n	Пузырёк / вставки	Пузырёк / шейкер	Шейкер / вставки
1000	11,3	5,7	2,0
2000	11,1	5,2	2,1
3000	13,1	5,9	2,2
4000	12,4	5,9	2,1
5000	12,4	5,9	2,1
6000	13,5	6,4	2,1

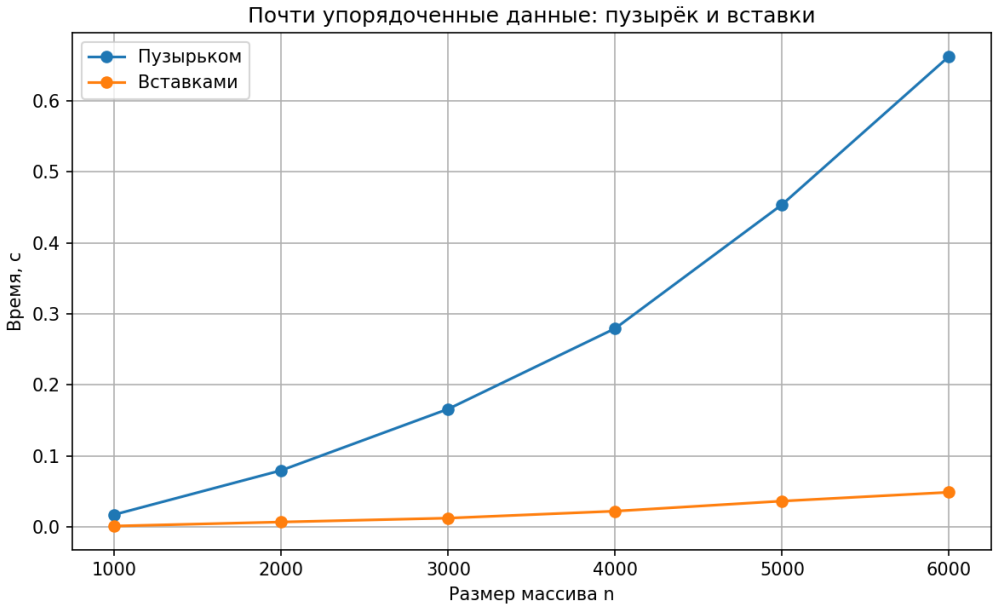


Рис. 5. Зависимость времени сортировки от размера массива: пузырёк и вставки

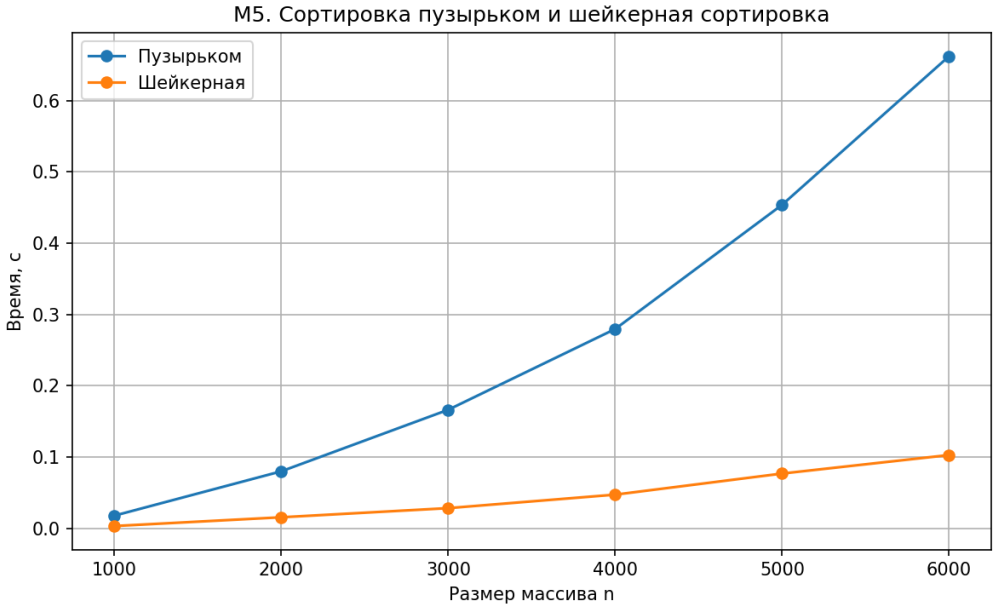


Рис. 6 (M5). Зависимость времени от n: сортировка пузырьком и шейкерная сортировка

7. Проверка квадратичного роста

Таблица 4. Отношение T(n)/n², ×10⁻⁹ с

n	Пузырьком	Шейкерная	Вставками
1000	17,68	3,10	1,57
2000	19,98	3,85	1,81
3000	18,48	3,14	1,41
4000	17,47	2,95	1,41
5000	18,16	3,07	1,47
6000	18,40	2,85	1,36

Таблица 5. Проверка квадратичного роста: T(2n)/T(n)

Переход	Пузырьком	Шейкерная	Вставками	Теория
1000 → 2000	4,52	4,96	4,60	4
2000 → 4000	3,50	3,06	3,12	4
3000 → 6000	3,98	3,64	3,86	4

Для квадратичной зависимости $T(n)/n^2$ должно быть приблизительно постоянным, а $T(2n)/T(n)$ — близким к 4. Оба условия выполняются для всех трёх алгоритмов.

Дополнительные измерения. Чтобы объяснить различия во времени, подсчитано число операций (функции `*_counted` в листинге; в замер времени они не входят).

Таблица 6. Число сравнений

n	Пузырьком	Шейкерная	Вставками	$n^2/2$
1000	478 794	51 569	32 446	500 000
2000	1 993 329	204 435	122 609	2 000 000
3000	4 459 160	441 524	278 634	4 500 000
4000	7 983 804	753 279	475 777	8 000 000
5000	12 496 834	1 137 739	724 907	12 500 000
6000	17 890 970	1 669 245	1 056 970	18 000 000

Таблица 7. Число обменов (для сортировки вставками — сдвигов)

n	Пузырьком	Шейкерная	Вставками
1000	31 447	31 447	31 447
2000	120 610	120 610	120 610
3000	275 635	275 635	275 635
4000	471 778	471 778	471 778
5000	719 908	719 908	719 908
6000	1 050 971	1 050 971	1 050 971

Таблица 8. Число проходов по массиву (для шейкерной — проходов в обоих направлениях)

n	Пузырьком	Шейкерная	Вставками
1000	796	53	999
2000	1 893	105	1 999
3000	2 719	151	2 999
4000	3 831	193	3 999
5000	4 963	233	4 999
6000	5 539	285	5 999

8. Выводы

1. Время работы всех трёх алгоритмов растёт квадратично. Отношение $T(n)/n^2$ почти постоянно: $17{,}5–20{,}0\cdot10^{-9}$ с у пузырька, $2{,}9–3{,}8\cdot10^{-9}$ с у шейкерной сортировки и $1{,}4–1{,}8\cdot10^{-9}$ с у вставок. При удвоении n время возрастает в 3,5–4,5 (пузырёк), 3,1–5,0 (шейкер) и 3,1–4,6 раза (вставки) при теоретическом значении 4. Отклонения вызваны линейными накладными расходами цикла, кэш-памятью процессора и случайными помехами при замере.
2. «Почти упорядоченные» данные (n/20 случайных перестановок пар) не дают линейного времени. Переставляемые элементы находятся в среднем на расстоянии около n/3 друг от друга, поэтому число инверсий (оно же число обменов, табл. 7) составляет около $0{,}030\cdot n^2$. Это примерно в 8 раз меньше, чем у случайного массива ($\approx n^2/4$), но всё равно квадратично

зависит от n. Поэтому сортировка вставками остаётся $\Theta(n^2)$, только с малым множителем.

3. Сортировка пузырьком выполняет 0,80–0,99·n проходов, то есть почти столько же, сколько в худшем случае. Причина: элемент, оказавшийся правее своего места, смещается влево лишь на одну позицию за проход. Поэтому флаг досрочного выхода почти не помогает, число сравнений составляет 96–100 % от $n^2/2$ (табл. 6). Из-за этого пузырёк в 11–13 раз медленнее вставок.
4. Шейкерная сортировка (M5) работает в 5,2–6,4 раза быстрее обычного пузырька. Обратный проход возвращает влево «лёгкие» элементы, попавшие вправо, гораздо быстрее, поэтому проходов требуется только около 0,05·n (порядка числа переставленных пар, n/20), а сравнений — в 9,3–11,0 раза меньше. Выигрыш по времени меньше, чем по сравнениям, потому что число обменов у всех трёх алгоритмов одинаково (оно равно числу инверсий), а обмен дороже сравнения.
5. Сортировка вставками остаётся самой быстрой из трёх: она в 2,0–2,2 раза быстрее шейкерной. Она делает меньше сравнений (1,6–1,7 раза меньше, чем шейкерная), а сдвиг (одно присваивание) дешевле обмена двух элементов. Для данных, близких к упорядоченным, это лучший из рассмотренных алгоритмов.
6. Все три алгоритма применимы лишь для небольших массивов. Экстраполяция по $T \approx c \cdot n^2$ даёт для n = 100 000: пузырёк — около 3 мин, шейкерная — около 32 с, вставки — около 15 с. Для больших объёмов нужны алгоритмы со сложностью $O(n \log n)$ (ЛР № 3).

Итог: эксперимент подтвердил теоретические оценки $\Theta(n^2)$ для всех трёх алгоритмов. На почти упорядоченных данных лучшей оказалась сортировка вставками; шейкерная сортировка существенно быстрее обычного пузырька, а флаг досрочного выхода пузырьку здесь почти не помогает.